



UNIVERSITÀ DI PARMA

Memory Allocation & Pointers

*Memoria est thesaurus omnium rerum et
custos*

De Oratore, Cicerone, I sec a.C.

- Limitations of arrays
- Variable Length Array
- Pointers
- void and NULL
- Functions for memory allocation
- Common Mistakes

SUMMARY



- So far we used fixed size arrays
 - we have to put dimension in code
- We must made assumptions on the amount of data to manage
 - Waste of memory
 - Bug → buffer overrun
- How to solve?
 - Variable Length Array (VLA)
 - Dynamic memory allocation





UNIVERSITÀ DI PARMA

VLA

Variable Length Arrays

- So far, we used constant to initialize arrays at **coding time**

```
int a[123];
```

- From C99 we can use variables to define the number of elements of an array at **runtime**

```
int size;  
scanf("%d", &size);  
int a[size];
```

- Coding time $\leftrightarrow$ Execution time
- No possible to initialize a VLA

- Up to C99 → constant length arrays
- C99 → also variable length arrays
- After C99 → maybe variable length arrays

- C++? → forked from C well before VLAs

- Not everyone is happy with VLAs
 - Linux Kernel is VLA free
- Pros
 - Simple syntax
- Cons
 - Portability (C++, C11...)
 - Allocated on stack

- VLAs also feature limitations
 - The size of arrays can not be modified
 - Automatic Duration
 - They are destroyed at the end of a code block
 - i.e. functions
 - I can not destroy them once no more needed
 - “Limited” size



UNIVERSITÀ DI PARMA

Pointers & Dynamic Memory Allocation

- Specific functions to dynamically allocate memory
 - Can be used to dynamically create arrays (or even single variables)
 - No automatic duration
 - They can be destroyed
- We need to introduce additional elements
 - Pointers
 - void type
 - size_t type

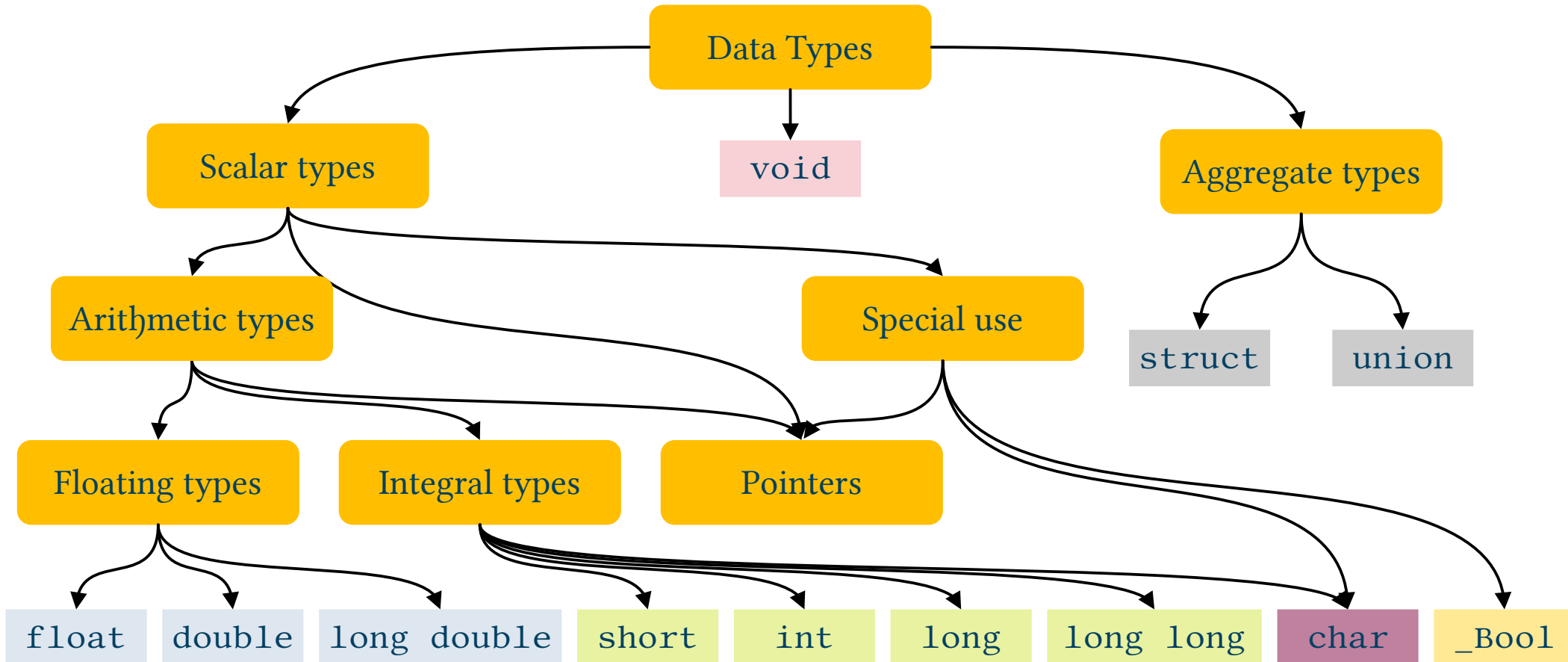
- Main functions

```
void *malloc(size_t size);  
void free(void *ptr);
```

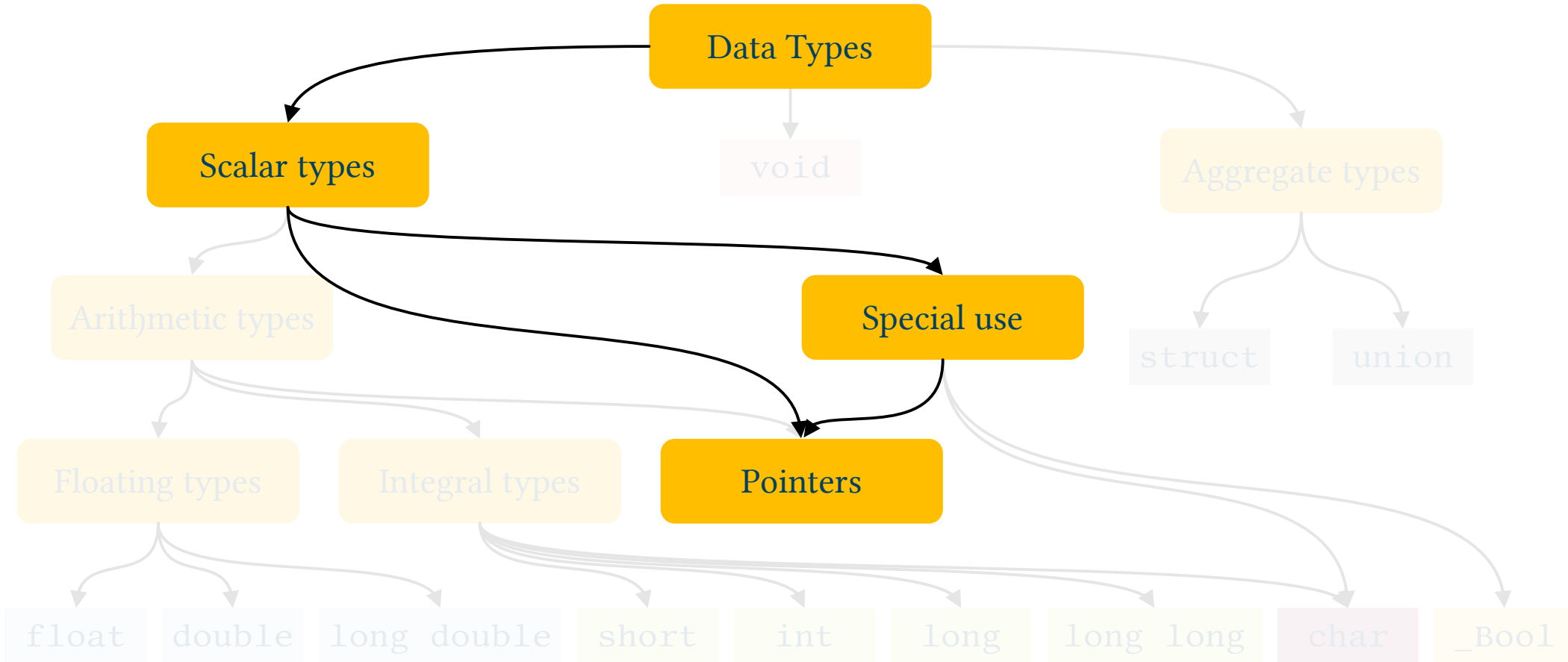
```
void *calloc(size_t nmemb, size_t size);  
void *realloc(void *ptr, size_t size);
```

- Something has not yet been discussed
 - **void? *? size_t?**

C99 Hierarchy of Data Types (simplified)



C99 Hierarchy of Data Types (simplified)



- Formally, scalar type
 - More precisely, integer numbers → special meaning
- Two main uses:
 - Dealing with dynamic memory management
 - We are going to discuss this in the following
 - To define functions that can have “side effects”
 - We will be back on this when speaking again about functions

- Variable → memory location with a name
 - Size
 - Address
- Address is data as well → in some cases I need to know it
- How to deal with addresses? → Pointers!
- A pointer is a **data type** meant to **store memory addresses**
 - Addresses of variables or even other data

- How to know the memory address of a variable?
 - & operators for single variables
 - Array name without [] for arrays
 - `printf("The address of variable a is %p\n", &a);`
 - `printf("The address of array n is %p\n", n);`
- How to store addresses? → a pointer variable:
 - `int a; // int type variable`
 - `int *x; // pointer to a int type`
 - `x = &a; // x now contains the address of a`

- To define a pointer-type variable use ‘*’ before the name

```
int *p;           // pointer to int data
```

```
float *x;         // pointer to float data
```

```
long a, *b;       // only b is a pointer!
```

- Given a pointer that contains the address of a data, how can I access that data?
- Using the '*' operator

```
int a;  
...  
int *x=&a;  
printf("At address %p, the stored int is %d\n", x, *x);
```

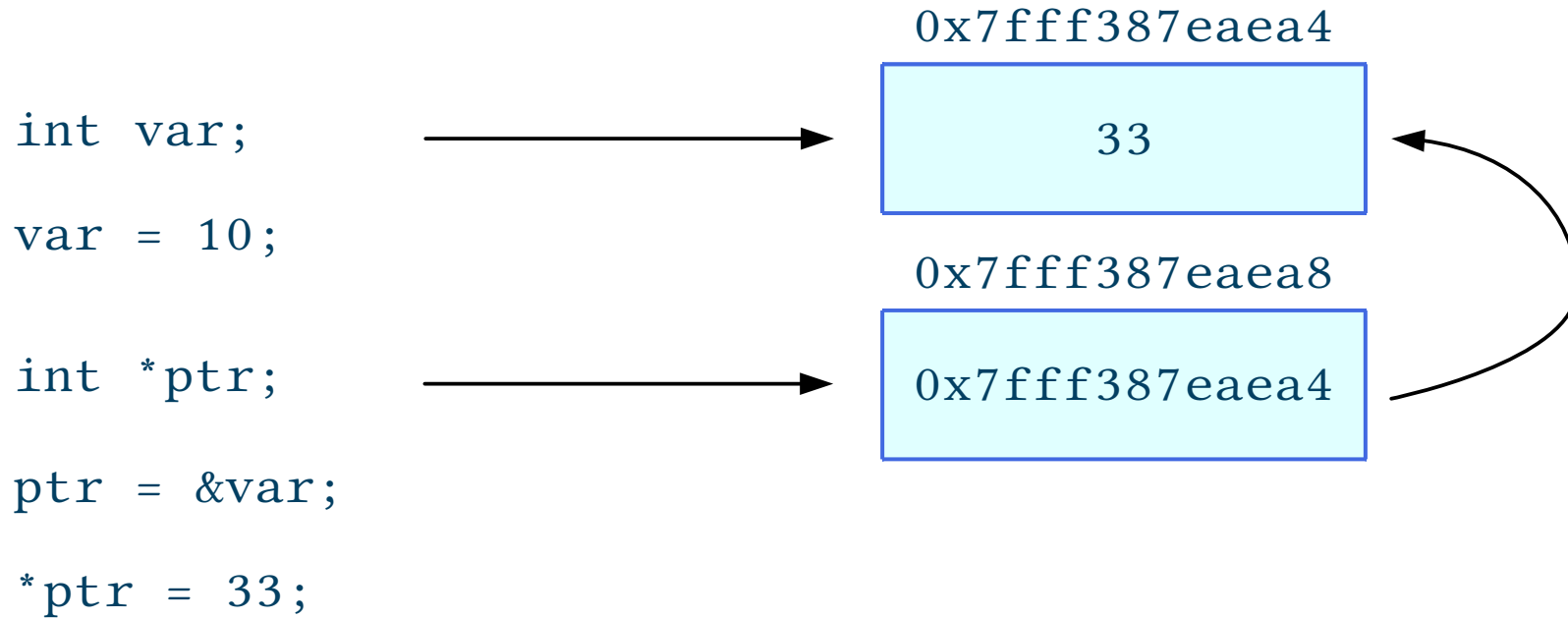
- Pointers and arrays share the same syntax!
- Arrays can be considered somewhat as constant pointers
- Specifically, **we can use [] also with pointers**
 - Do it only when a pointer contains a valid address

```
int a[100] = { something }; // define and init an array
```

```
int pa = a; // pa now contains the address of array a
```

```
pa[33] = somevalue; // we can use pa to deal with that “buffer”
```

Pointers: recap

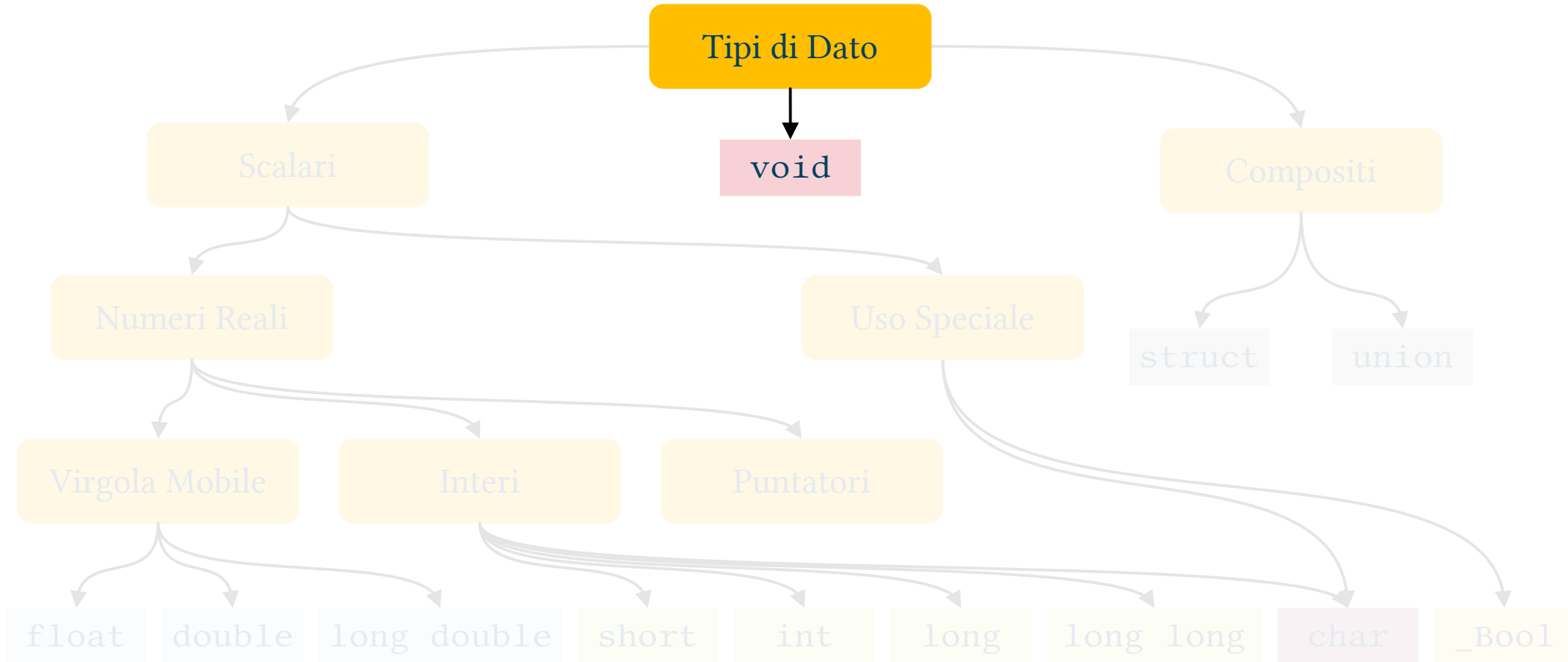


- How we can deal with a function that takes pointers as args?
 - We have already been using some of them
 - `scanf()` → `scanf("%d", &a);`
 - We need to pass an address either using `&` or directly a pointer
- How it is defined?
 - `*` is again involved
 - i.e. `strlen(const char *s);`

- Functions can also return pointers
- How it is defined?
 - ‘*’ is again involved, example:
 - **char** *strchr(const char *s, int c);
 - This function returns the address of a char
 - We must, anyway, specify the type of data at which the address refer to

- By definition an invalid memory address
- Typical uses:
 - Functions returning an address can return NULL when they fails or when they have no result to return
 - i.e. search for a given data
 - To init a pointer variable
 - As argument for some functions

C99 Hierarchy of Data Types (simplified)



- Is a *no-type*
- Not possible to define *void* variables
- Used for:
 - Functions
 - No returned values
 - No parameters
 - Pointers
 - Generic type

- Alias for another data type
- The type capable to store the maximum memory address that we can have to deal with
 - Depends on given architecture
 - Usually in 64 bits systems: *unsigned long long* or *unsigned long*
 - Data type returned by `sizeof()`
 - To print it `%zt` (C99, for our workstations → `%ld`)

- Now we have all elements that allow to understand dynamic memory primitives
 - Pointers and their syntax
 - * to define
 - * also to access referenced data
 - [] to access data in a “buffer” like an array
 - How to pass them to functions as arguments
 - How functions can return an address
 - void data type
 - size_t

- Main functions:

```
void *malloc(size_t size);
```

```
void free(void *ptr);
```

```
void *calloc(size_t nmemb, size_t size);
```

```
void *realloc(void *ptr, size_t size);
```

- Now syntax should be clear!

```
void *malloc(size_t size);
```

- It allows to allocate a block of memory
 - Input → how many bytes do I need
 - Returned value → Address of allocated memory
 - NULL on error, namely when it is not possible to perform the requested memory allocation
- Memory is not initialized

- malloc() returns an address → a pointer variable can be used to store it
 - I should check whether NULL has been returned
- When the address is ok, how to access that block of memory?
 - Do you remember that pointers and arrays share the same syntax?
 - **We can use [] also with pointers**

- C does not feature a “Garbage Collector”
 - When a dynamically allocated block of memory is no more needed I have to manually “release” it
 - Otherwise that block is considered in use **forever**
- The free() function performs this
- Input → address of a dynamically allocated block of memory
- Output → nothing



```
void *calloc(size_t nmemb, size_t size);
```

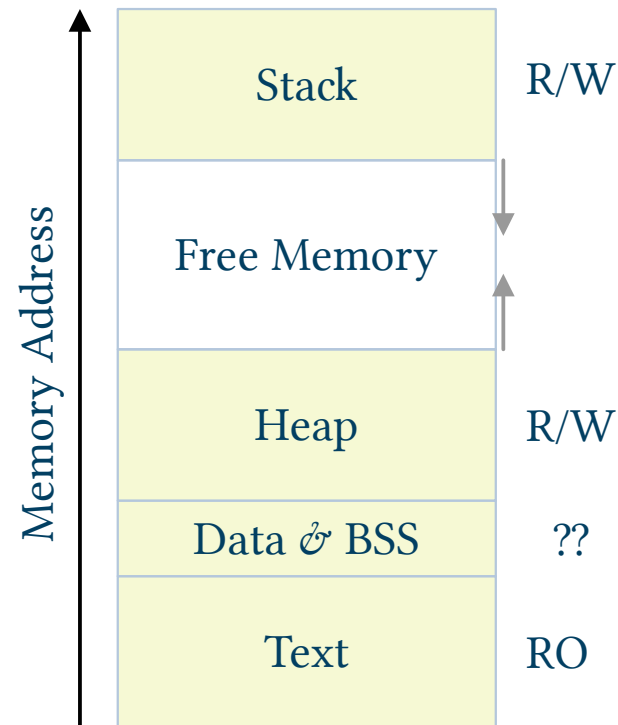
- Same as malloc() but memory is set to 0
 - Input → number of elements and their size
 - Returned value → address of allocated “array”
 - NULL when calloc() fails
 - All elements are zeroed

`void *realloc(void *ptr, size_t size);`

- It allows to change the size of a dynamically allocated block of memory
 - Input → address of a buffer to resize & new size
 - Data are copied into the new block of memory
 - Part of them when reducing the size
 - Returned value → address of the “new” block of memory
 - Or NULL when resize is not possible

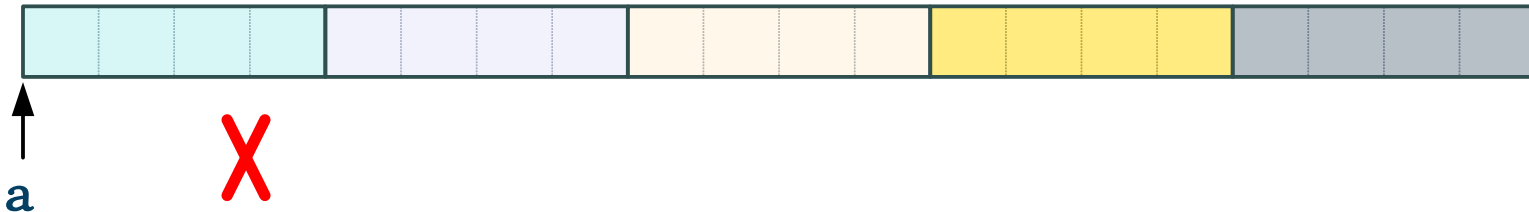
Memory layout of a C program

- Many sections
 - Stack (.stack) → local variables, VLA
 - Last In First Out approach
 - Heap (.heap) → allocated blocks (malloc()...)
 - Code (.text) → code
 - Data → global or static variables
 - .data & .bss



- Formally pointers are a Scalar Type
 - We can use $+$ and $-$ operators and variants ($++$, $--$, $+= \dots$)
- The result depends on the size of the type of data pointers are referencing
- Incrementing by x
 - It does not mean adding x *bytes* to the address stored in the pointer
 - It means incrementing the address by x *positions*, as going over an array

- If a pointer is referencing the `<type>` data type, adding `x` means adding `x` times the byte size of `<type>` data type
 - Used only when the pointer is referencing an array
- The `a[x]` syntax is basically the same, at low level
 - $a[x] \rightarrow *(a + x)$
 - Example: given a pointer `int *a` we want to add 3 to that pointer (remember that `sizeof(int)==4`)



- Easy to have bugs when dealing with memory addresses!
 - Buffer size
 - Lifetime of variables
 - Loss of addresses



- Already discussed for arrays
- Easy to go beyond the range of a block of memory
 - After or even before
- In addition → NULL or not initialized pointer



- malloc()/calloc() allow to reserve a block of memory
 - This enable us to access such area
- And if I “lose” that address?
 - That area can not be used anymore
 - But still “reserved”
- When we have this issue:
 - Assignment errors
 - Automatic variables



- Roughly the opposite of the previous case
 - Address is still known
- I do not lose the address but I can no more access that memory area
 - Namely, that memory block is no more “reserved” for us
- Causes
 - Automatic variables
 - Mistakes in using `free()`





UNIVERSITÀ DI PARMA

Pointers to Pointers

- Also pointer variables feature a memory address
- Can we store that address?
- Yes → pointers to pointers
- You need to use a double `**` in definitions
- i.e. `int **p2p; //p2p is a double pointer`
- Also pointer to pointer variables feature a memory address...
 - You can use a lot of `'*'`...



Pointers to Pointers: syntax

```
int var;
```

```
var = 10;
```

```
int *ptr;
```

```
ptr = &var;
```

```
*ptr = 33;
```

```
int **dptr;
```

```
dptr = &ptr;
```

```
**dptr = 57; // note the double **
```

0x7fff387eaea4

57

0x7fff387eaea8

0x7fff387eaea4

0x7fff387eaeb0

0x7fff387eaea8



- Do you remember the main() function?
 - `int main(int argv, char **argc);`
- Why we need them?
 - When dealing with functions that have side effects, or
 - In order to dynamically allocate blocks of memory
 - Often when managing multidimensional arrays





UNIVERSITÀ DI PARMA

Dynamic allocation of multidimensional arrays

- For sake of simplicity, just consider a bi-dimensional array
- A statically allocated 2D array can be defined as:

```
float b[33][55];
```

- To access a specific element, we can use a syntax like:

```
b[17][34] = ... // we MUST use [][] and indexes
```

- To access array address

```
printf("the address is %p\n", b) // we have to omit  
[]
```

- Can we use a single index with a bi-dimensional array?
 - i.e., it is correct to use and what is the meaning of
`b[17]` ??
- When we omit some dimension, this always represents an address. Which one?
- We can see `b` as an **array of arrays**, namely `b[17]` is the address of the 18th row
`printf("the address of 18th row is %p\n", b[17])`

- 6 different possibilities:
 - 1) **1D array with pointer arithmetic**
 - 2) *VLA of pointers*
 - 3) **Double pointer with a malloc() for each row**
 - 4) *Double pointer and a single malloc()*
 - 5) *A pointer to a VLA*
 - 6) *A pointer to the first row of a VLA*

1D array with pointer arithmetic

- When we want to manage a matrix with ***r* rows** and ***c* columns**
- Simply allocate a block of ***r*c* elements**, i.e.,

```
float *matr = malloc(r*c*sizeof(float));
```
- The element in row ***y*** and column ***x*** can be easily accessed as

```
matr[x + y*c] = ...
```
- Pros → easy allocation (1 malloc())
- Cons → we need to use a different syntax for access + math

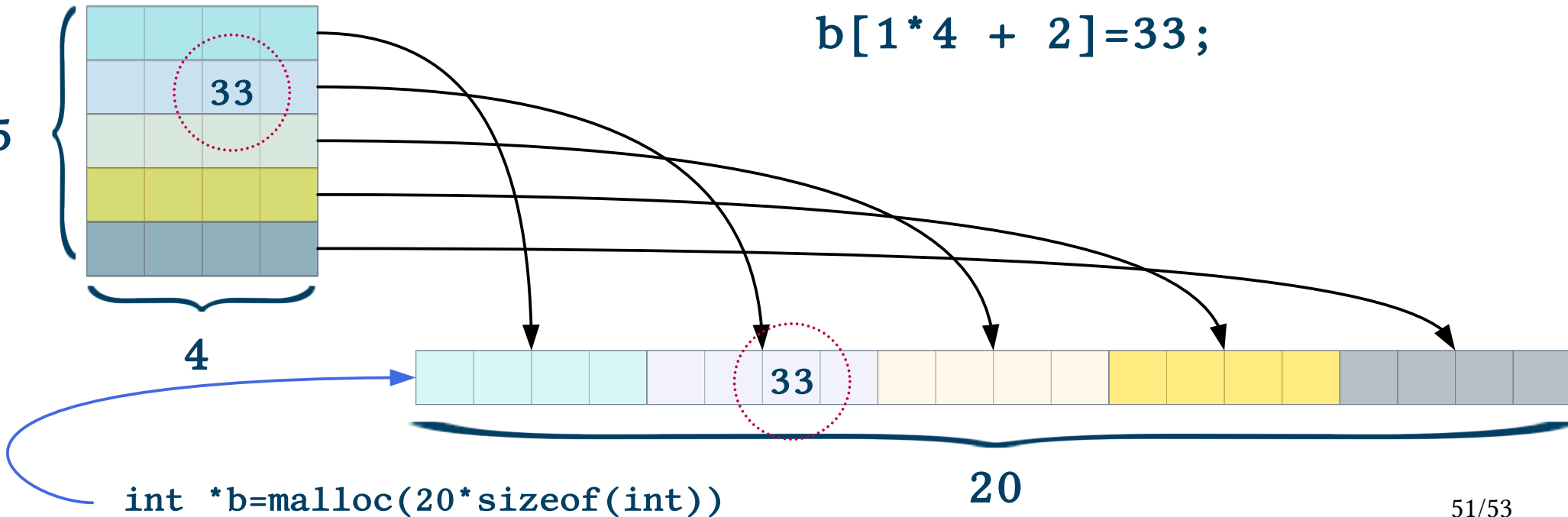
1D array with pointer arithmetic

- Logical vs “Physical” layout

`a[1][2]=33;`

`int a[5][4]`

`b[1*4 + 2]=33;`



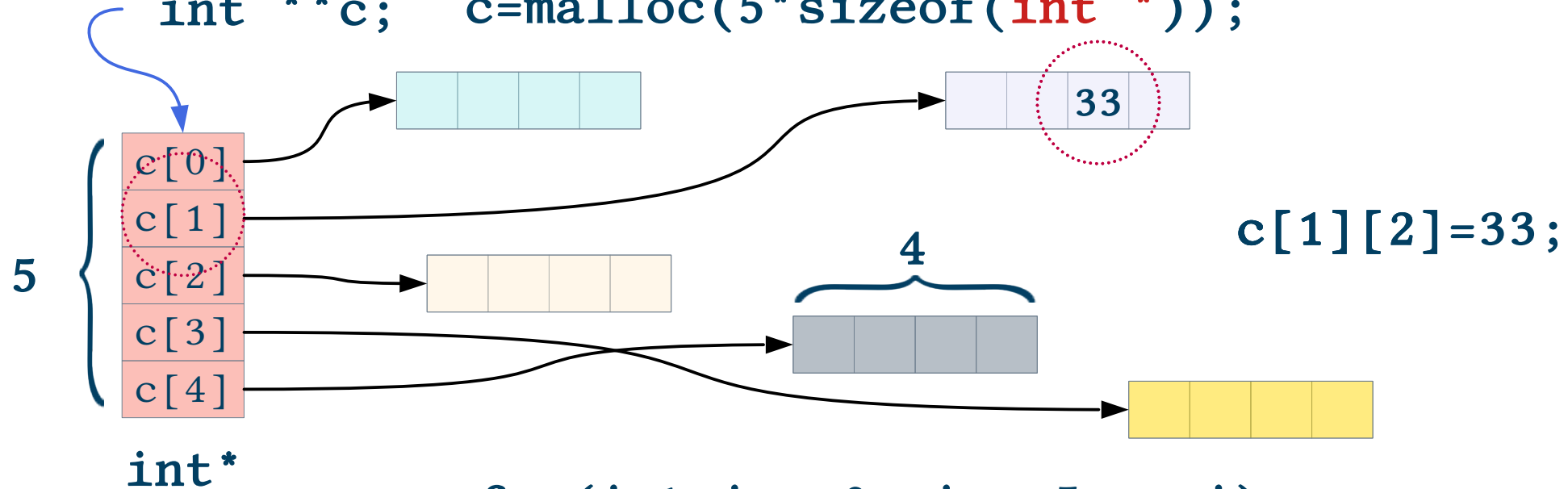
Double pointer with a malloc() for each row

- When we want to manage a matrix with ***r* rows** and ***c* columns**
- A double pointer can be used to allocate *r* “row” pointers, one for each row
- Each row is then independently allocated
- Pros:
 - Same syntax as multidim arrays, i.e., `[][]`
 - Can also manage different sized rows
- Cons: complex memory allocation & destroy

Double pointer with a malloc() for each row

- As before we want to dynamically allocate a 5×4 matrix

```
int **c;  c=malloc(5*sizeof(int *));
```



```
for(int i = 0; i < 5; ++i)  
    c[i] = malloc(4*sizeof(int));
```



UNIVERSITÀ DI PARMA

Memory Allocation & Pointers



KEEP
CALM
IT'S
QUESTION
TIME

Do we pay for bytes? No, so what...

Z, VisLab srl, 2014

- Limitations of arrays
- Variable Length Array
- Pointers
- void and NULL
- Functions for memory allocation
- Common Mistakes

SUMMARY



- So far we used fixed size arrays
 - we have to put dimension in code
- We must made assumptions on the amount of data to manage
 - Waste of memory
 - Bug → buffer overrun
- How to solve?
 - Variable Length Array (VLA)
 - Dynamic memory allocation





- So far, we used constant to initialize arrays at **coding time**

```
int a[123];
```

- From C99 we can use variables to define the number of elements of an array at **runtime**

```
int size;  
scanf("%d", &size);  
int a[size];
```

- Coding time $\leftrightarrow$ Execution time
- No possible to initialize a VLA

- Up to C99 → constant length arrays
- C99 → also variable length arrays
- After C99 → maybe variable length arrays

- C++? → forked from C well before VLAs



- Not everyone is happy with VLAs
 - Linux Kernel is VLA free
- Pros
 - Simple syntax
- Cons
 - Portability (C++, C11...)
 - Allocated on stack



- VLAs also feature limitations
 - The size of arrays can not be modified
 - Automatic Duration
 - They are destroyed at the end of a code block
 - i.e. functions
 - I can not destroy them once no more needed
 - “Limited” size



- Specific functions to dynamically allocate memory
 - Can be used to dynamically create arrays (or even single variables)
 - No automatic duration
 - They can be destroyed
- We need to introduce additional elements
 - Pointers
 - void type
 - size_t type



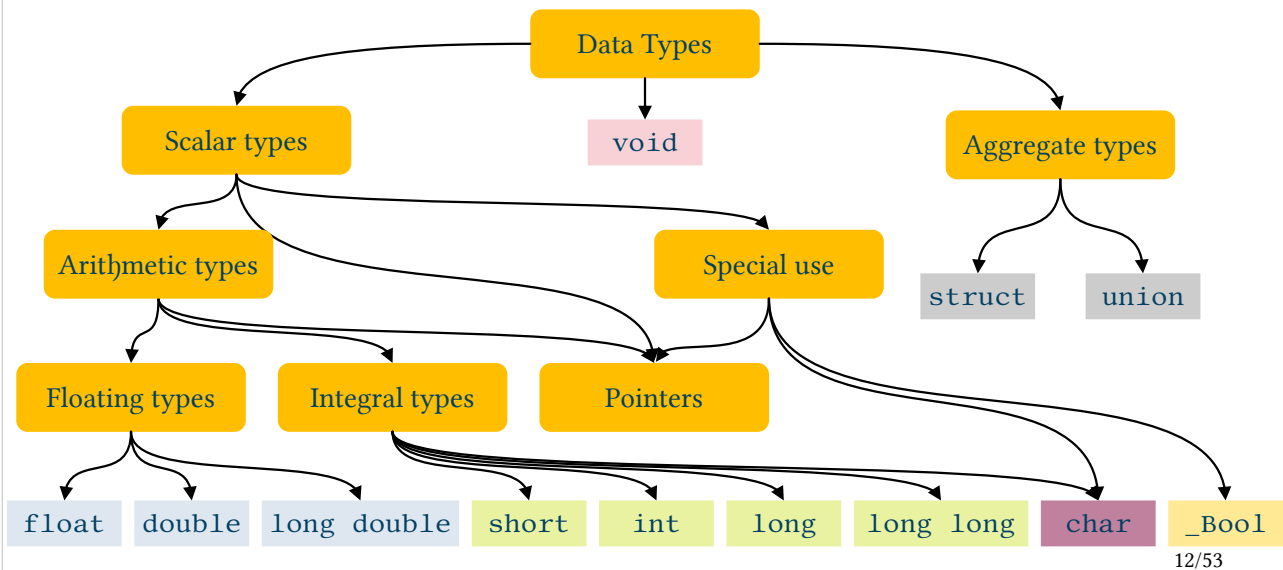
- Main functions

```
void *malloc(size_t size);  
void free(void *ptr);
```

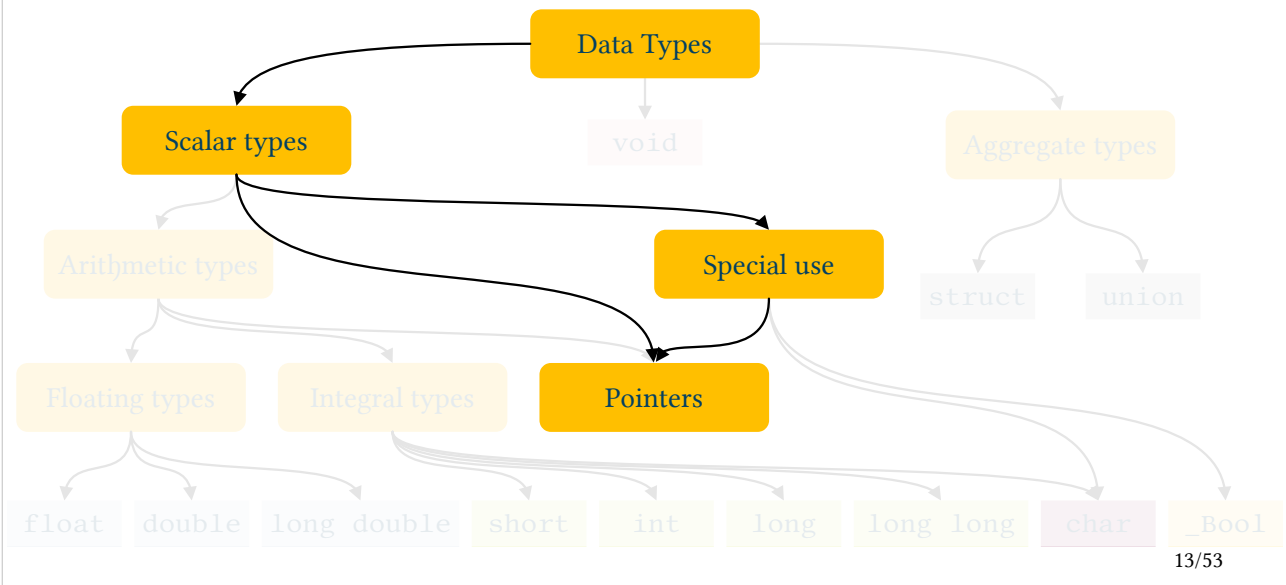
```
void *calloc(size_t nmem, size_t size);  
void *realloc(void *ptr, size_t size);
```

- Something has not yet been discussed
 - **void? *? size_t?**

C99 Hierarchy of Data Types (simplified)



C99 Hierarchy of Data Types (simplified)





- Formally, scalar type
 - More precisely, integer numbers → special meaning
- Two main uses:
 - Dealing with dynamic memory management
 - We are going to discuss this in the following
 - To define functions that can have “side effects”
 - We will be back on this when speaking again about functions



- Variable → memory location with a name
 - Size
 - Address
- Address is data as well → in some cases I need to know it
- How to deal with addresses? → Pointers!
- A pointer is a **data type** meant to **store memory addresses**
 - Addresses of variables or even other data

- How to know the memory address of a variable?
 - & operators for single variables
 - Array name without [] for arrays
 - `printf("The address off variable a is %p\n", &a);`
 - `printf("The address of array n is %p\n", n);`
- How to store addresses? → a pointer variable:
 - `int a; // int type variable`
 - `int *x; // pointer to a int type`
 - `x = &a; // x now contains the address of a`



- To define a pointer-type variable use '*' before the name

```
int *p;          // pointer to int data
```

```
float *x;        // pointer to float data
```

```
long a, *b;      // only b is a pointer!
```



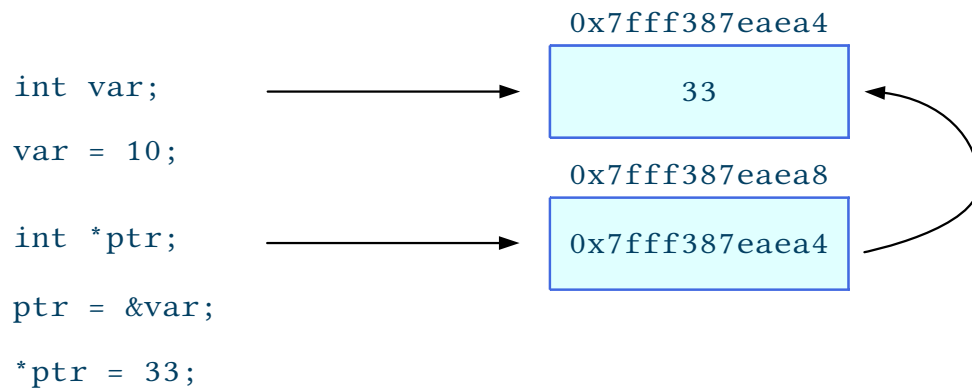
- Given a pointer that contains the address of a data, how can I access that data?
- Using the '*' operator

```
int a;  
...  
int *x=&a;  
printf("At address %p, the stored int is %d\n", x, *x);
```

- Pointers and arrays share the same syntax!
- Arrays can be considered somewhat as constant pointers
- Specifically, **we can use [] also with pointers**
 - Do it only when a pointer contains a valid address

```
int a[100] = { something }; // define and init an array
int pa = a;                  // pa now contains the address of array a
pa[33] = somevalue;         // we can use pa to deal with that "buffer"
```

Pointers: recap



- How we can deal with a function that takes pointers as args?
 - We have already been using some of them
 - `scanf()` → `scanf("%d", &a);`
 - We need to pass an address either using `&` or directly a pointer
- How it is defined?
 - ‘*’ is again involved
 - i.e. `strlen(const char *s);`

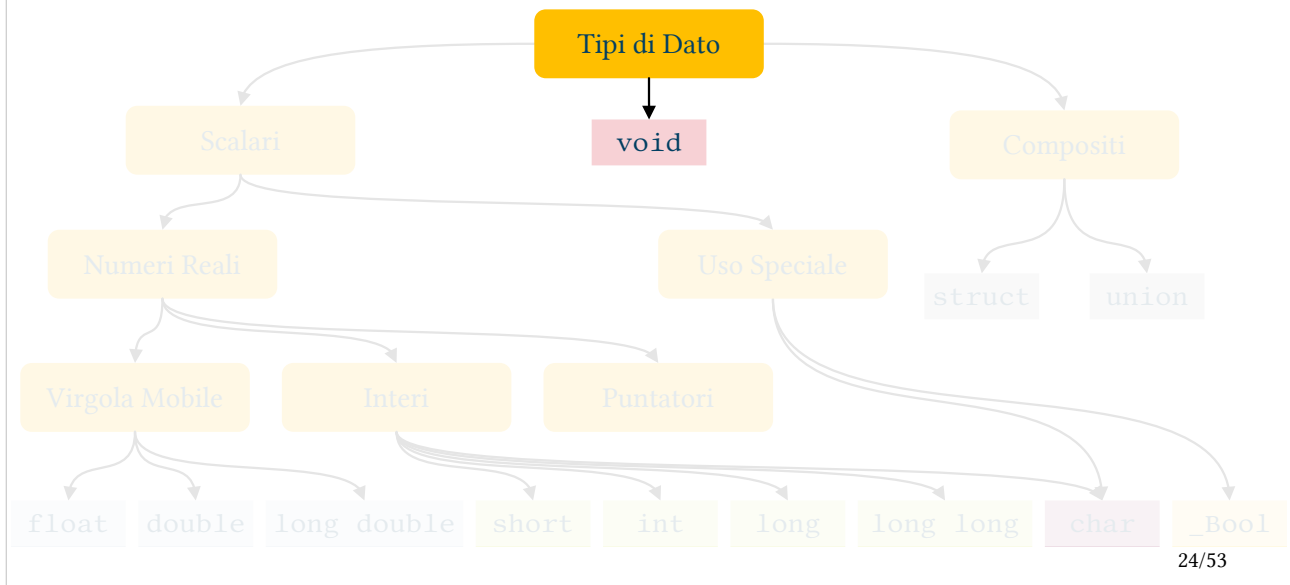


- Functions can also return pointers
- How it is defined?
 - ‘*’ is again involved, example:
 - **char** *strchr(const char *s, int c);
 - This function returns the address of a char
 - We must, anyway, specify the type of data at which the address refer to



- By definition an invalid memory address
- Typical uses:
 - Functions returning an address can return NULL when they fails or when they have no result to return
 - i.e. search for a given data
 - To init a pointer variable
 - As argument for some functions

C99 Hierarchy of Data Types (simplified)



- Is a *no-type*
- Not possible to define *void* variables
- Used for:
 - Functions
 - No returned values
 - No parameters
 - Pointers
 - Generic type

- Alias for another data type
- The type capable to store the maximum memory address that we can have to deal with
 - Depends on given architecture
 - Usually in 64 bits systems: *unsigned long long* or *unsigned long*
 - Data type returned by `sizeof()`
 - To print it `%zt` (C99, for our workstations → `%ld`)

- Now we have all elements that allow to understand dynamic memory primitives
 - Pointers and their syntax
 - * to define
 - * also to access referenced data
 - [] to access data in a “buffer” like an array
 - How to pass them to functions as arguments
 - How functions can return an address
 - void data type
 - size_t



- Main functions:

```
void *malloc(size_t size);  
void free(void *ptr);
```

```
void *calloc(size_t nmem, size_t size);  
void *realloc(void *ptr, size_t size);
```

- Now syntax should be clear!



- It allows to allocate a block of memory
 - Input → how many bytes do I need
 - Returned value → Address of allocated memory
 - NULL on error, namely when it is not possible to perform the requested memory allocation
- Memory is not initialized



- malloc() returns an address → a pointer variable can be used to store it
 - I should check whether NULL has been returned
- When the address is ok, how to access that block of memory?
 - Do you remember that pointers and arrays share the same syntax?
 - **We can use [] also with pointers**

- C does not feature a “Garbage Collector”
 - When a dynamically allocated block of memory is no more needed I have to manually “release” it
 - Otherwise that block is considered in use **forever**
- The `free()` function performs this
- Input → address of a dynamically allocated block of memory
- Output → nothing



```
void *calloc(size_t nmemb, size_t size);
```



- Same as malloc() but memory is set to 0
 - Input → number of elements and their size
 - Returned value → address of allocated “array”
 - NULL when calloc() fails
 - All elements are zeroed

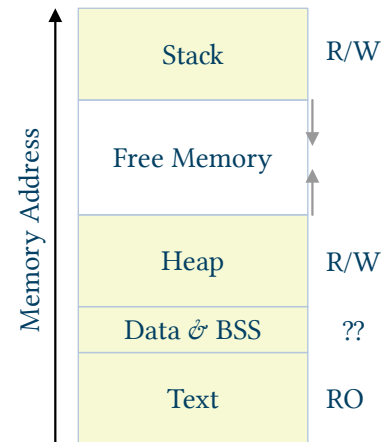
```
void *realloc(void *ptr, size_t size);
```



- It allows to change the size of a dynamically allocated block of memory
 - Input → address of a buffer to resize & new size
 - Data are copied into the new block of memory
 - Part of them when reducing the size
 - Returned value → address of the “new” block of memory
 - Or NULL when resize is not possible



- Many sections
 - Stack (.stack) → local variables, VLA
 - Last In First Out approach
 - Heap (.heap) → allocated blocks (malloc(...))
 - Code (.text) → code
 - Data → global or static variables
 - .data & .bss

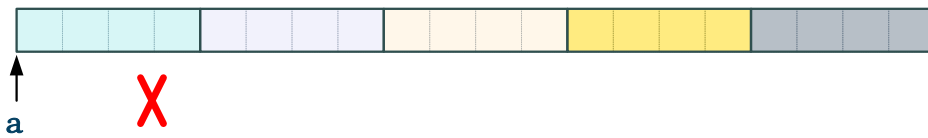




- Formally pointers are a Scalar Type
 - We can use + and – operators and variants (++ , -- , += ...)
- The result depends on the size of the type of data pointers are referencing
- Incrementing by x
 - It does not mean adding x bytes to the address stored in the pointer
 - It means incrementing the address by x positions, as going over an array



- If a pointer is referencing the `<type>` data type, adding `x` means adding `x` times the byte size of `<type>` data type
 - Used only when the pointer is referencing an array
- The `a[x]` syntax is basically the same, at low level
 - `a[x] → *(a + x)`
 - Example: given a pointer `int *a` we want to add 3 to that pointer (remember that `sizeof(int)==4`)



- Easy to have bugs when dealing with memory addresses!
 - Buffer size
 - Lifetime of variables
 - Loss of addresses





- Already discussed for arrays
- Easy to go beyond the range of a block of memory
 - After or even before
- In addition → NULL or not initialized pointer



- malloc()/calloc() allow to reserve a block of memory
 - This enable us to access such area
- And if I “lose” that address?
 - That area can not be used anymore
 - But still “reserved”
- When we have this issue:
 - Assignment errors
 - Automatic variables



- Roughly the opposite of the previous case
 - Address is still known
- I do not lose the address but I can no more access that memory area
 - Namely, that memory block is no more “reserved” for us
- Causes
 - Automatic variables
 - Mistakes in using `free()`

- Also pointer variables feature a memory address
- Can we store that address?
- Yes → pointers to pointers
- You need to use a double `**` in definitions
- i.e. `int **p2p; //p2p is a double pointer`
- Also pointer to pointer variables feature a memory address...
 - You can use a lot of `*`...



Pointers to Pointers: syntax



```
int var;
var = 10;

int *ptr;
ptr = &var;
*ptr = 33;

int **dptr;
dptr = &ptr;
**dptr = 57; // note the double **
```

0x7fff387eaea4

57

0x7fff387eaea8

0x7fff387eaea4

0x7fff387eae00

0x7fff387eaea8





- Do you remember the main() function?
 - `int main(int argv, char **argc);`
- Why we need them?
 - When dealing with functions that have side effects, or
 - In order to dynamically allocate blocks of memory
 - Often when managing multidimensional arrays





- For sake of simplicity, just consider a bi-dimensional array
- A statically allocated 2D array can be defined as:
`float b[33][55];`
- To access a specific element, we can use a syntax like:
`b[17][34] = ... // we MUST use [][] and indexes`
- To access array address
`printf("the address is %p\n", b) // we have to omit []`



- Can we use a single index with a bi-dimensional array?
 - i.e., it is correct to use and what is the meaning of
`b[17]` ??
- When we omit some dimension, this always represents an address. Which one?
- We can see `b` as an **array of arrays**, namely `b[17]` is the address of the 18th row
`printf("the address of 18th row is %p\n", b[17])`



- 6 different possibilities:
 - 1) **1D array with pointer arithmetic**
 - 2) *VLA of pointers*
 - 3) **Double pointer with a malloc() for each row**
 - 4) *Double pointer and a single malloc()*
 - 5) *A pointer to a VLA*
 - 6) *A pointer to the first row of a VLA*



- When we want to manage a matrix with ***r* rows** and ***c* columns**
- Simply allocate a block of ***r*c* elements**, i.e.,
`float *matr = malloc(r*c*sizeof(float));`
- The element in row ***y*** and column ***x*** can be easily accessed as
`matr[x + y*c] = ...`
- Pros → easy allocation (1 malloc())
- Cons → we need to use a different syntax for access + math

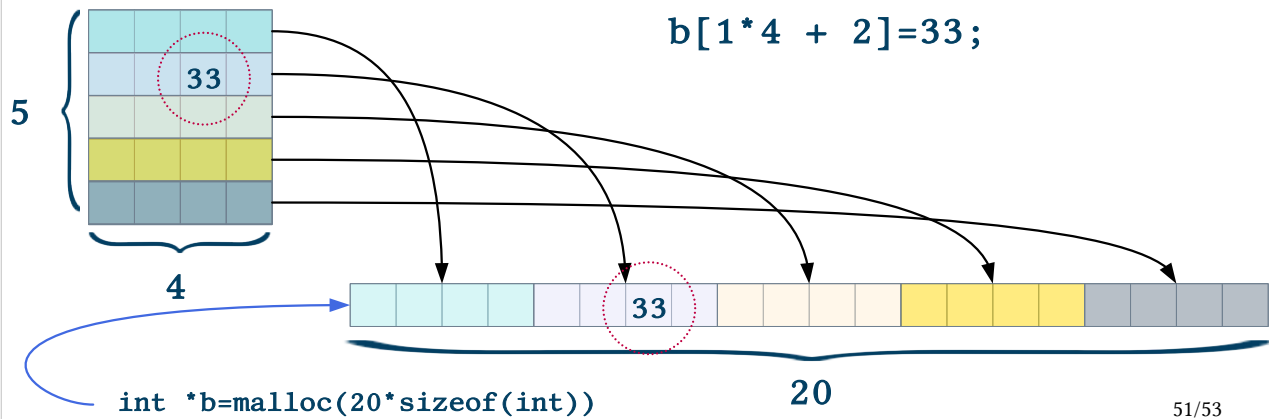
1D array with pointer arithmetic

- Logical vs “Physical” layout

`a[1][2]=33;`

`int a[5][4]`

`b[1*4 + 2]=33;`



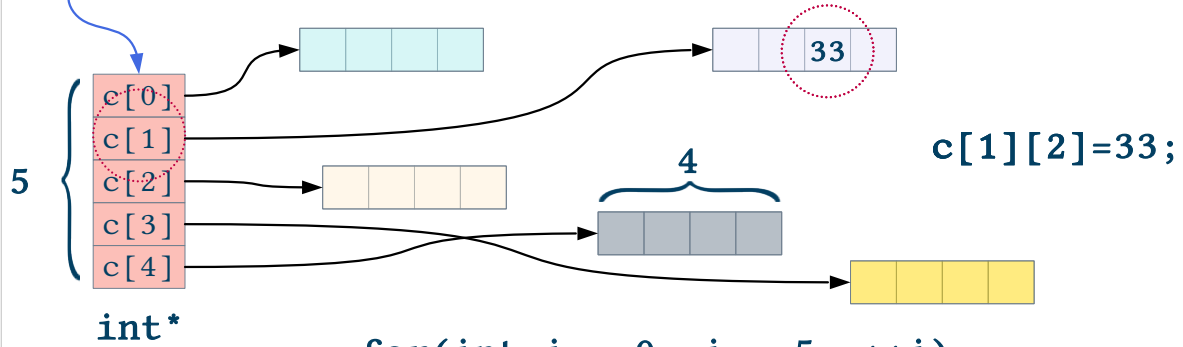


- When we want to manage a matrix with ***r* rows** and ***c* columns**
- A double pointer can be used to allocate *r* “row” pointers, one for each row
- Each row is then independently allocated
- Pros:
 - Same syntax as multidim arrays, i.e., `[][]`
 - Can also manage different sized rows
- Cons: complex memory allocation & destroy

Double pointer with a malloc() for each row

- As before we want to dynamically allocate a 5×4 matrix

```
int **c; c=malloc(5*sizeof(int *));
```



```
for(int i = 0; i < 5; ++i)  
    c[i] = malloc(4*sizeof(int));
```

